CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: BarberConnect

Prepared By: Cabasag, Ma. Melessa Villas

Version: 3.0.0

Date: 02/05/2026
Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0.0	02/02/2026	Cabasag, Ma. Melessa Villas	Initial draft	Draft
1.2.0	02/11/2026	Cabasag, Ma. Melessa Villas	Added requirements and functional requirements	Review
2.0.0	02/03/2026	Cabasag, Ma. Melessa Villas	Added Admin(USER), ERD	Revise
3.0.0	03/05/2026	Cabasag, Ma. Melessa Villas	refined architecture, corrected component diagram references, updated technology stack, revised DB schema for Firestore NoSQL, corrected API contracts	Draft

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5
3.0 NON-FUNCTIONAL REQUIREMENTS 8
4.0 SYSTEM ARCHITECTURE 9
5.0 API CONTRACT & COMMUNICATION 10
Authentication Endpoints 11
User Registration 11
User Login 11
6.0 DATABASE DESIGN 13
7.0 UI/UX DESIGN 14
8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

BarberConnect is a multi-platform barbershop management system that connects Customers, Barbers, Admin within a centralized digital platform. The system automates appointment scheduling, haircut pricing, income distribution (80/20 split), analytics reporting, social interaction (posts, comments, reactions), and administrative monitoring.

The system uses:

- Supabase (PostgreSQL) Database
- Java 17, Spring Boot 3.x, Spring Security, Spring Data
- Firebase (Firestore, Firebase Auth, Cloud Storage, FCM)
- React 18, TypeScript, Tailwind CSS, Axios
- Kotlin, Jetpack Compose, Retrofit, Firebase SDK
- Railway/Render (Backend), Vercel/Netlify (Web), APK (Mobile)

1.2 Objectives

1. Develop a secure multi-role barber booking platform
2. Implement smart scheduling and availability management
3. Automate 80% barber / 20% owner income distribution
4. Provide analytics dashboards for barbers and admin
5. Enable social engagement features (posts, reactions, comments) via Firestore collections.

1.3 Scope

Included Features

- Role-based authentication (Customer, Barber, Admin) via Firebase Auth
- Smart appointment booking with conflict prevention
- Haircut styles with base price and add-on styling options
- Cash and E-Cash (GCash) payment options
- Automatic 80/20 income computation per completed appointment

- Feedback and rating system
- Post, comment, and reaction social feed
- Barber leave and availability management
- Admin monitoring and income tracking dashboard

Excluded Features

- Credit/Debit online payment gateway
- SMS notification system
- AI haircut recommendation

1.0 INTRODUCTION

1.1 Purpose

This document defines the architecture, functional requirements, non-functional requirements, API design, database schema, and development plan for BarberConnect. It ensures that all system components are properly integrated and aligned with business objectives.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: BarberConnect
Domain: Service Booking & Business Management
Primary Users:

1. Customer
2. Barber
3. Admin

Problem Statement: Manual scheduling and income tracking in traditional barbershops are inefficient and lack transparency.
Solution: A centralized digital platform that automates booking, pricing, income distribution, and analytics using Firebase as the BaaS layer

2.2 Core User Journeys

Journey 1: Customer Booking Flow

1. Customer registers/logs in
2. Customer registers or logs in via Firebase Authentication.
3. Browses the barber directory and filters by preference or rating.

4. Selects a barber and views available haircut styles.
5. Chooses styling add-on options with additional pricing.
6. Picks an available date and time slot.
7. Selects payment method (Cash or E-Cash/GCash).
8. Confirms booking; Firestore appointment document is created.
9. Leaves a feedback rating after the service is completed.

Journey 2: Barber Management Flow

1. Barber logs in via Firebase Authentication.
2. Sets availability schedule or marks leave days.
3. Adds/edits haircut styles and pricing in their catalog.
4. Views upcoming scheduled customers.
5. Posts haircut photos to the social feed; replies to comments.
6. Views income analytics dashboard (monthly income, feedback percentage).
7. Updates profile details (name, email, password, phone, GCash number).

Income Distribution Logic:

- 80% of service fee → Barber
- 20% of service fee → Owner

Journey 3: Admin Monitoring Flow

1. Admin logs in with a privileged Firebase Auth account.
2. Sets and enforces haircut price range policies.
3. Monitors all barber schedules (Free / Booked / Leave).
4. Tracks total accumulated owner income (20% share).
5. Reviews individual barber performance metrics

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication via Firebase Auth (register, login, logout, password reset)
2. Multi-role system (Customer, Barber, Admin) using Firestore custom claims
3. Smart appointment scheduling with conflict prevention
4. Haircut catalog with base price and styling options
5. Automated 80/20 income distribution on booking completion
6. Feedback and rating system per completed appointment
7. Barber analytics dashboard
8. Admin monitoring dashboard

SHOULD HAVE

1. Barber filtering by name and rating
2. Post reaction system (like/love/etc.)
3. Leave tracking dashboard for admin

COULD HAVE

1. Advanced analytics charts
2. Performance ranking system

WON'T HAVE

1. Real-time bank integration
2. AI haircut recommendation

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields**: Email, Password, Confirm Password, Role Selection (Customer/Barber)
- **Validation**: Email format validation, password strength (min 8 characters), uniqueness of email
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, POST /auth/forgot-password
- **Security**: Firebase ID Token (JWT), custom claims for role (RBAC), server-side token verification via Spring Security

Feature: Haircut & Service Catalog

- **Screens:** Barber Profile, Haircut Listing, Haircut Detail
- **Display:** Haircut image (stored in Firebase Storage), style name, base price, styling options with additional price
- **Search:** By barber name or rating
- **API Endpoints:** GET /barbers, GET /barbers/{id}, POST /haircuts, PUT /haircuts/{id}, DELETE /haircuts/{id}
- **Admin Functions:** Set minimum and maximum allowed haircut price range via admin_settings Firestore document

Feature: Appointment Scheduling

- **Screens:** Booking Page, Schedule Calendar, Appointment Summary

- **Functions:** Select barber, haircut style, styling options, date and time slot
- **Validation:** Prevent booking during leave days or already booked time slots (checked server-side via Firestore transaction)
- **Persistence:** Appointment document stored in Firestore under /appointments collection
- **API Endpoints:** GET /appointments, POST /appointments, PUT /appointments/{id}, DELETE /appointments/{id}

Feature: Payment & Income Distribution

- **Screens:** Payment Selection, Booking Confirmation
- **Data Collected:** Cash, E-Cash (GCash)
- **Process:** Compute total price (base + add-ons), apply 80/20 split, write income_record document to Firestore
- **API Endpoints:** POST /payments, GET /income/barber/{id}, GET /income/owner
- **Business Logic:** barberShare = totalPrice * 0.80 | ownerShare = totalPrice * 0.20

Feature: Social Interaction & Feedback

- **Screens:** Dashboard Feed, Post Detail
- **Functions:** Post haircut photos to Firestore /posts, react (like/love), comment, reply
- **Ratings:** Customer gives star rating + comment after completed appointment
- **API Endpoints:** POST /posts, POST /comments, POST /reactions, POST /feedback

Feature: Admin & System Monitoring

- **Screens:** Admin Dashboard, Income Overview, Schedule Monitoring
- **Functions:** View all barber availability, monitor cumulative owner income, manage price range policy
- **Access Control:** Admin role enforced via Firebase custom claim + Spring Security RBAC
- **API Endpoints:** GET /admin/schedules, GET /admin/income, PUT /admin/settings

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter valid email and strong password
- ➢ And confirm password matches
- ➢ And click "Create Account"

- Then Firebase Auth creates my account and issues a JWT ID token
- And I am automatically logged in and redirected to the homepage

AC-2: Successful Appointment Booking

- Given I am logged in as a Customer
- When I select a barber
- And choose a haircut style with options
- And select an available date and time
- And choose a payment method
- And confirm booking
- Then a Firestore appointment document is created
- And the booking appears in the barber's schedule
- And the booking should appear in the barber schedule

AC-3: Admin Product Management

- Given I am logged in as a barber
- When I set my schedule to 'Leave' in my availability
- Then customers should not be able to book on that date
- And the leave should reflect in my monthly leave count

AC-4: Admin Income Monitoring

- Given I am logged in as an admin
- When I view income dashboard
- Then I should see total owner income (20%)
- And individual barber earnings (80%) are listed per barber

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Firestore reads/writes complete in ≤ 500 ms under normal load
- Support 100 concurrent users without degradation
- Income computation processed atomically (Firestore transaction) immediately after booking completion

3.2 Security Requirements

- All network traffic over HTTPS/TLS
- Firebase Authentication issues signed JWT ID tokens; all API calls verify token via Firebase Admin SDK
- Firestore Security Rules enforce per-document read/write permissions by role
- Spring Security RBAC guards all REST endpoints
- Admin endpoints require ADMIN custom claim on Firebase token
- Sensitive user data (passwords) never stored in Firestore — managed exclusively by Firebase Auth

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 major versions)
- **Android:** API Level 24+ (Android 7.0 Nougat and above)
- **Screen sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems for web:** Windows 10+, macOS 10.15+, Ubuntu 20.04+

3.4 Usability Requirements

- Entire booking flow completable in under 3 minutes
- Clear barber availability indicators: Free / Booked / Leave
- Descriptive validation error messages with recovery hints
- Responsive UI on both web and Android mobile

3.5 Reliability

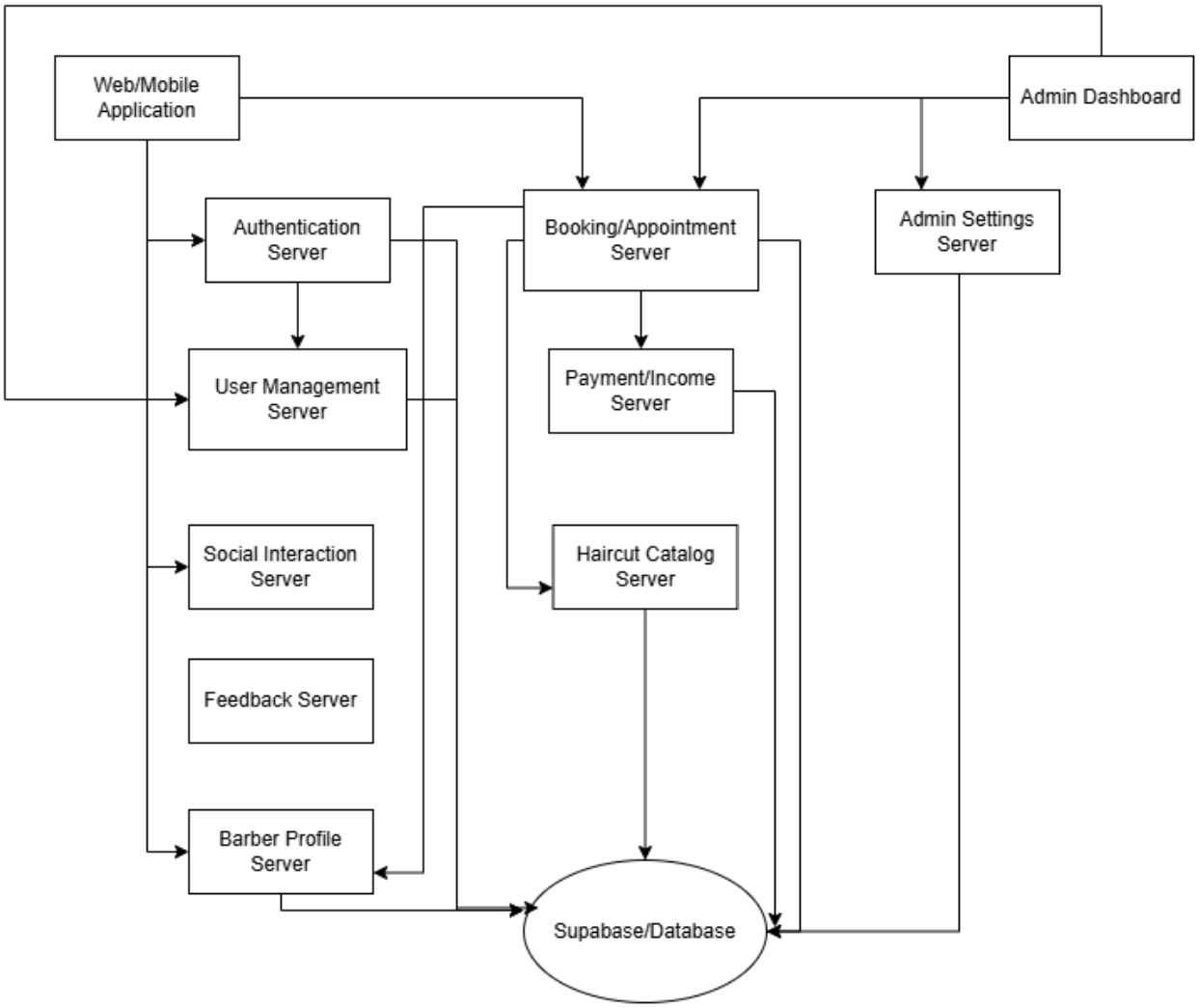
- Target 99% uptime during the testing phase (leveraging Firebase's SLA)
- Booking and income computation use Firestore atomic transactions to prevent race conditions
- Consistent Firestore data integrity via server-side validation in Spring Boot

3.6 Scalability

- Firestore auto-scales horizontally with document volume
- Modular Spring Boot microservice-style API structure
- Firebase Cloud Storage scales automatically for media assets
- Architecture designed for future multi-branch extension

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data, Firebase Admin SDK
- **Database:** Firebase Authentication, Cloud Firestore, Cloud Storage for Firebase, Firebase Cloud Messaging (FCM)
- **Web Frontend:** Re React 18, TypeScript, Tailwind CSS, Axios, Firebase JS SDK
  Mobile: Kotlin, Jetpack Compose, Retrofit, Firebase Android SDK (Auth, Firestore, Storage, Messaging)
- **Build Tools:** Maven (Backend), npm / yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Firebase ID Token (JWT) passed as Bearer token in Authorization header. The Spring Boot backend verifies the token using the Firebase Admin SDK.
Response Structure	Firebase custom claims (role: CUSTOMER | BARBER | ADMIN) are embedded in the ID token and read by Spring Security for RBAC
Standard Response Envelope	{ "success": boolean, "data": object | null, "error": { "code": string, "message": string, "details": object | null }, "timestamp": string // ISO 8601 }

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Creates a new Firebase Auth account and writes a user document to Firestore /users with the assigned role.
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "<email>", "password": "<password>", "firstname": "<firstname>", "lastname": "<lastname>", "role": "CUSTOMER | BARBER" }
Response Structure	{

	"success": true, "data": { "uid": "<firebase_uid>", "email": "...", "accessToken": "<idToken>", "refreshToken": "<refreshToken>" } }

User Login

Description	Authenticates user via Firebase Auth REST API; returns Firebase ID token and refresh token.
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "<email>", "password": "<password>" }
Response Structure	{ "success": true, "data": { "uid": "...", "email": "...", "role": "CUSTOMER|BARBER|ADMIN", "accessToken": "<idToken>", "refreshToken": "<refreshToken>" } }

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource successfully created
- 400 Bad Request - Invalid input or missing required fields
- 401 Unauthorized - Firebase ID token missing, expired, or invalid
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Requested resource does not exist in Firestore
- 500 Internal Server Error - Unhandled server error

Error Code Examples

Example 1	json {

	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

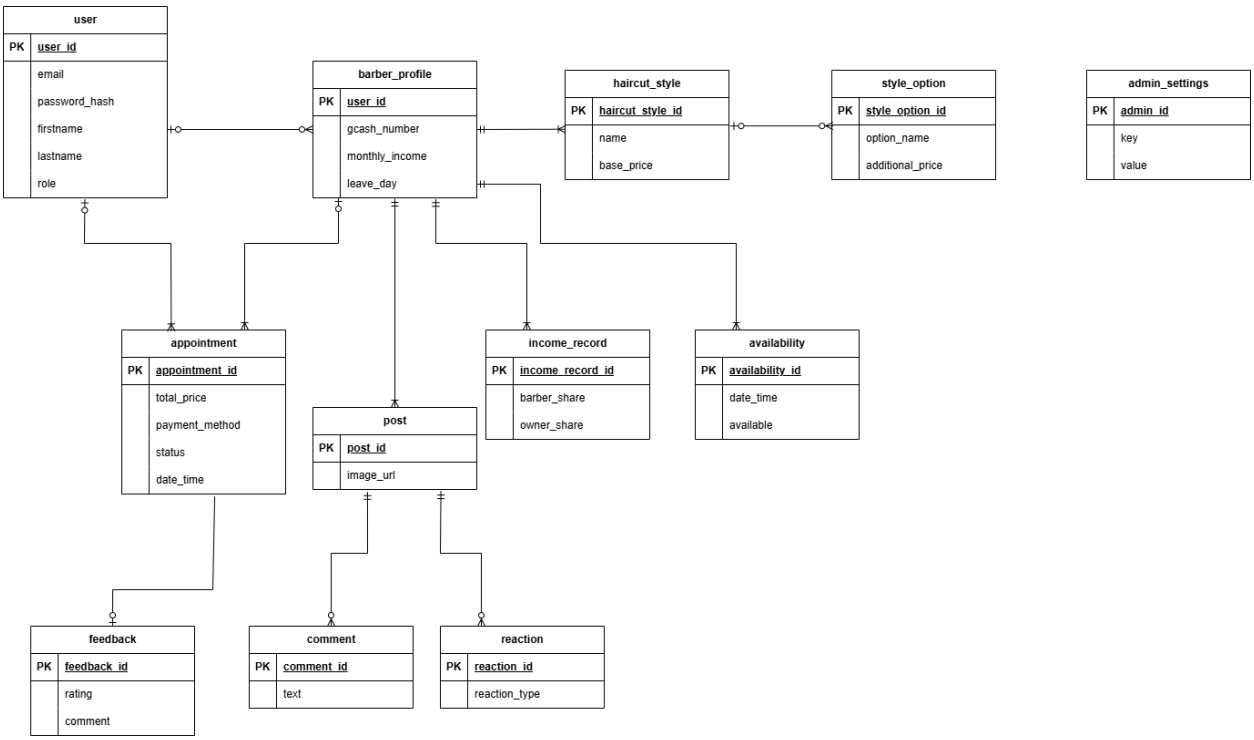
Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-One:** user ↔ barber_profile (Each barber user has exactly one barber profile)
- **One-to-Many:** user → appointment (Customers can book multiple appointments)
- **One-to-Many:** barber_profile → haircut_style (Each barber can define multiple haircut style)
- **One-to-Many:** haircut_style → style_option(Each haircut style can have multiple styling options)
- **One-to-many: barber_profile → income_record (Each barber has multiple income records over time)**
- **Many-to-One:** appointment → barber_profile (Each appointment references a barber)**Many-to-One:** appointment→ user (Each appointment references a customer)
- **Many-to-one:** income_record → barber_profile (Each income record belongs to a barber)

Key Tables:

1. **user** - Stores authentication and role information

2. **barber_profile** – Holds barber-specific details(Gcash, income, leave days)
3. **haircut_style** – Catalog of haircut styles per barbers
4. **style_option** – Additional styling options linked to haircut styles
5. **appointment** – Customer booking with barber, service details, and payment method
6. **feedback** - Customer rating and comment after an appointment.
7. **income_record** – Tracks income distribution (80% baber, 20% owner)
8. **admin_settings** – Stores system-wide policies (pricing rules, monitoring settings).

Table Structure Summary:

- **user:** user_id, email, password_hash, role (CUSTOMER, BARBER, ADMIN)
- **baber_profile:** user_id, gcash_number, monthly_income, leave_days
- **haicut_style:** id, barber_id, name, base_price
- **style_option:** id, haircut_id, option_name, additional_price
- **appoinment:** id, customer_id, barber_id, total_price, payment_method, status
- **income_record:** id, barber_id, barber_share (80%), owner_share (20%)
- **admin_settings –** admin_id, key, value

7.0 UI/UX DESIGN

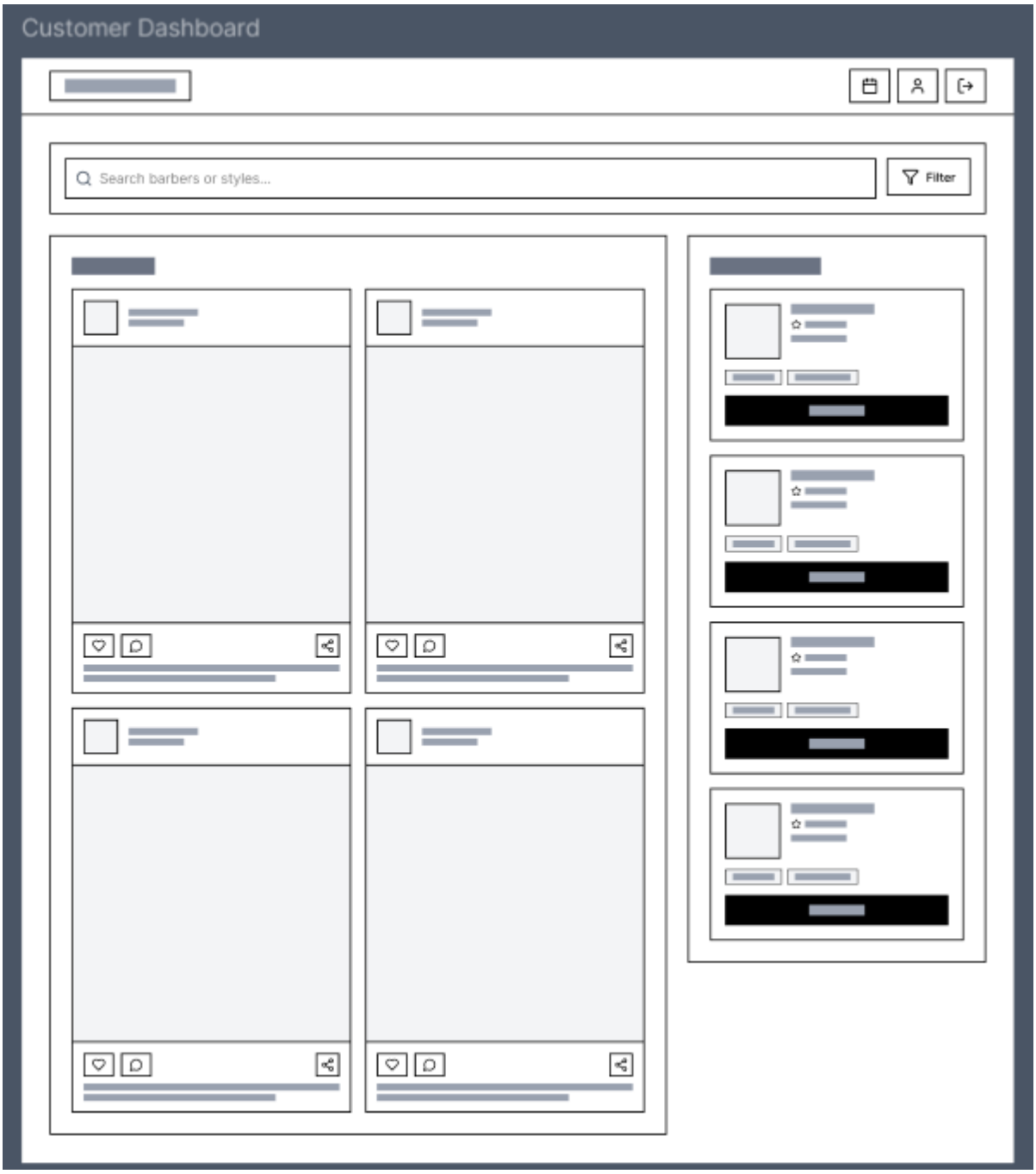
7.1 Web Application Wireframes

Login:

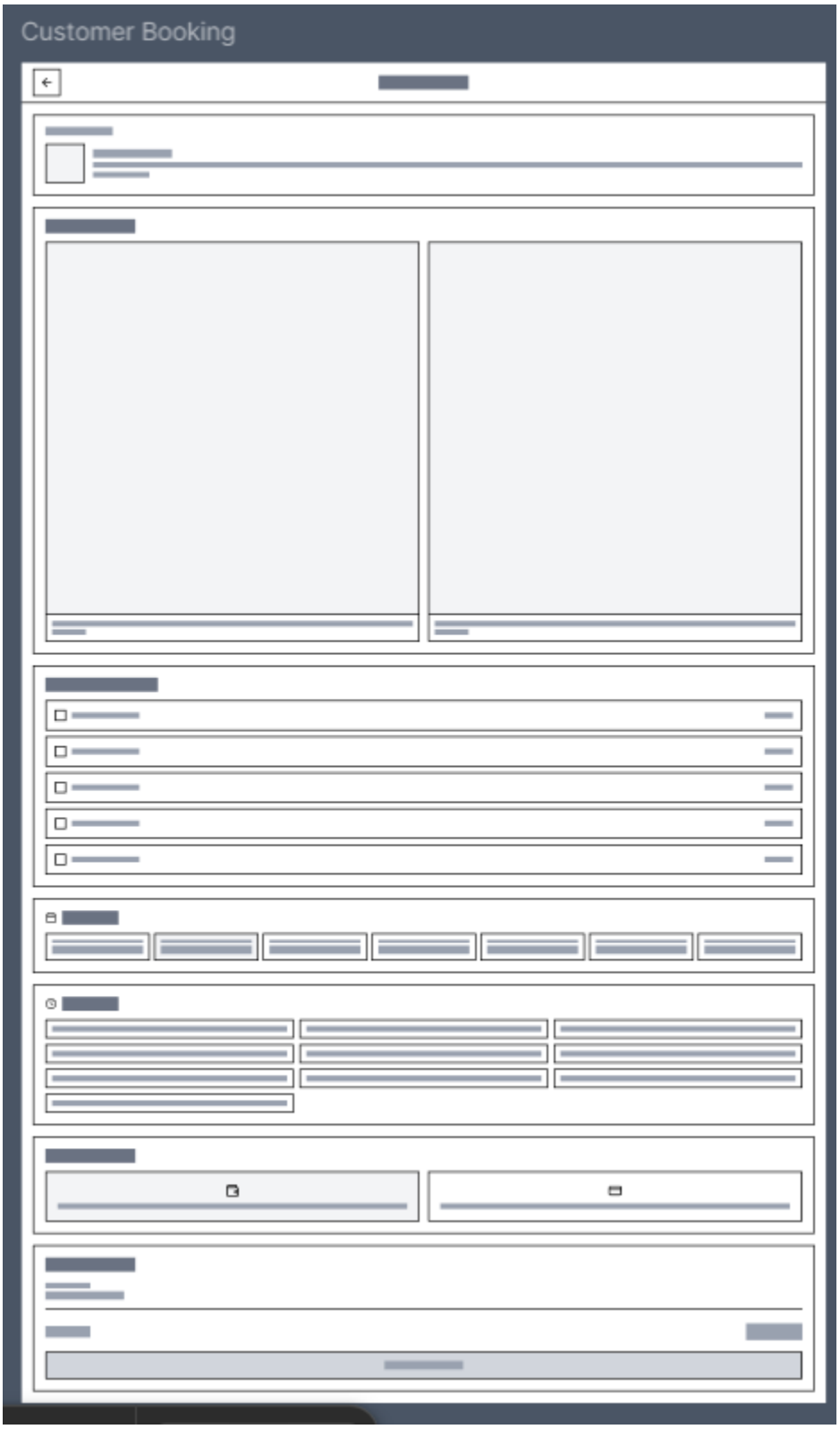
Register:

ForgotPassword

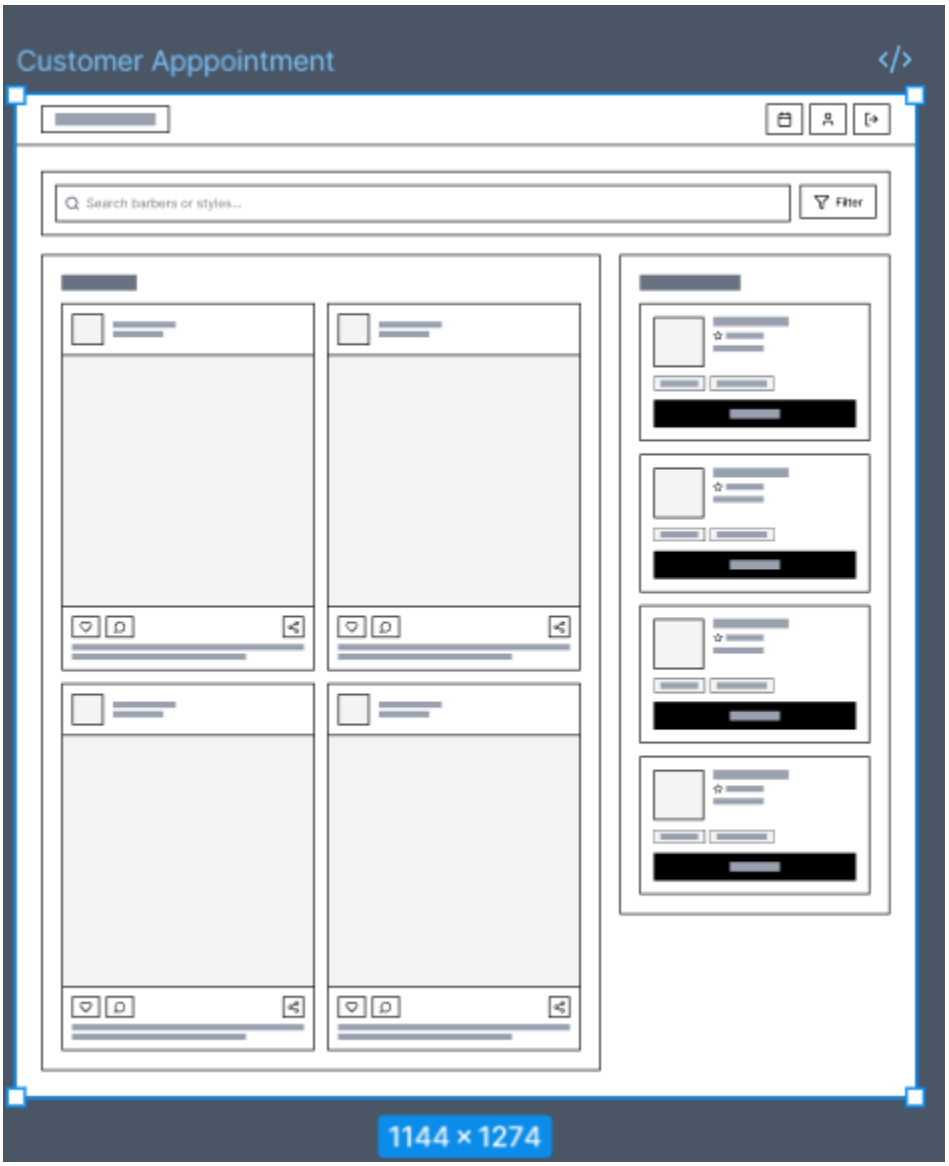
Customer Dashboard



Customer Booking



Customer Appointment



Customer Profile Page

Barber Dashboard

Barber Profile Page

Admin Dashboard Page

7.2 Mobile Application Wireframes

Login Page

Register

Forgot Password

Customer Dashboard , Booking and Appointment

Customer Profile Barber Dashboard Barber Profile

Admin Dashboard

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1-2: Spring Boot setup with dependencies

Database configuration and entities

Day 3: Authentication

JWT authentication implementation

Day 4: Authentication & Role System

Implement registration

Implement login

JWT generation

Role-Based Access Control (CUSTOMER, BARBER, ADMIN)

Secure protected endpoints

Day 5: Barber & Haircut Module

- Create barber profile table
- CRUD haircut styles
- Basic validation

Simple Booking System (NO advanced logic yet)

Week 4: Core Business Logic

Day 1: Improve Booking Logic

Day 2: Income Computation

Day 3: Feedback & Rating

Day 4: Basic Admin Panel (Backend only)

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Web Application (Frontend Only Now)

Day 1: Setup frontend

Connect to backend

Login page

Day 2: Authentication pages (login, register)

Day 3: Barber listing page

Show haircut styles

Day 4: Booking page

Select haircut

Select date/time

Show computed total

Day 5: Basic dashboard Implementation both(Customer and Barber)

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Improve validation messages

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Barber Browsing

Day 4: Booking Screen

Day 5: View Appointments

Week 8: Improve & Polish

Day 1: Barber View

Day 2: Rating Feature

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All planning and design documents completed (FRS, NFR, Database Design, API Specification, UI/UX Wireframes)
- **M2 (End Week 4):** Backend API fully functional (Authentication, Role-Based Access Control, Booking System, Income Computation, Feedback System)
- **M3 (End Week 6):** Web application complete (Full booking flow, dashboards for Customer & Barber, Admin monitoring)
- **M4 (End Week 8):** Mobile application complete (Authentication, Barber browsing, Booking, Appointment viewing, Rating feature)
- **M5 (End Week 10):** Full system deployed, integrated, and ready for final presentation

Critical Path:

1. Authentication & JWT Security (Week 3)
2. Booking conflict validation logic (Week 4)
3. 80/20 income computation implementation (Week 4)
4. Full booking flow integration in Web application (Week 6)
5. Cross-platform integration and end-to-end testing (Week 9)

Risk Mitigation:

- Start with the simplest working version of each feature before adding enhancements
- Test booking and income computation logic early to avoid major rework
- Keep regular backups of backend and database schema
- Focus on core booking functionality before UI improvements

- Avoid adding extra features beyond the defined project scope